

GovMind

An autonomous AI governance operator for DAOs: how the agent works, and how Modal and Pydantic power it

Agent	Pydantic AI Agent on Google Gemini (gemini-3.5-flash), 24 tools
Channels	Telegram (DAO group chat, private chats, Mini App) and WhatsApp Cloud API
Runtime	Modal: web endpoint, cron sweep, volume, secrets
Typing	Pydantic v2 end to end: tool inputs, results, webhooks, settings
On-chain	Endless Chain: balances, EDS transfers, immutable audit records
Live	https://driftypencil--govmind-web.modal.run · t.me/GovMind_bot

1. What GovMind does

DAOs control large treasuries, but governance runs on apathy: fewer than 10% of token holders vote, burn rate and runway go unwatched, and attackers can quietly accumulate tokens to pass a proposal that drains the treasury. GovMind is one agent that lives where members already talk, in the DAO's Telegram group, and does four jobs:

- **Proposal intelligence.** Reads each proposal, calculates its treasury impact, profiles the proposer's wallet (age, funding source, history), rates the risk, and posts a structured briefing.
- **Vote mobilisation.** Finds members who haven't voted and sends each a personal nudge. It never tells anyone how to vote.
- **Treasury health.** Tracks balances, burn rate, runway and concentration risk, raises alerts when thresholds are crossed, and sends charts.
- **Attack detection.** Looks for coordinated wallet funding, token accumulation before votes, fresh wallets that vote, and suspicious proposals, then posts a GOVERNANCE ALERT with evidence.

Important actions (creating proposals, casting votes, alerts) are also written to Endless Chain as an audit trail, and the reply includes an explorer link.

2. System architecture

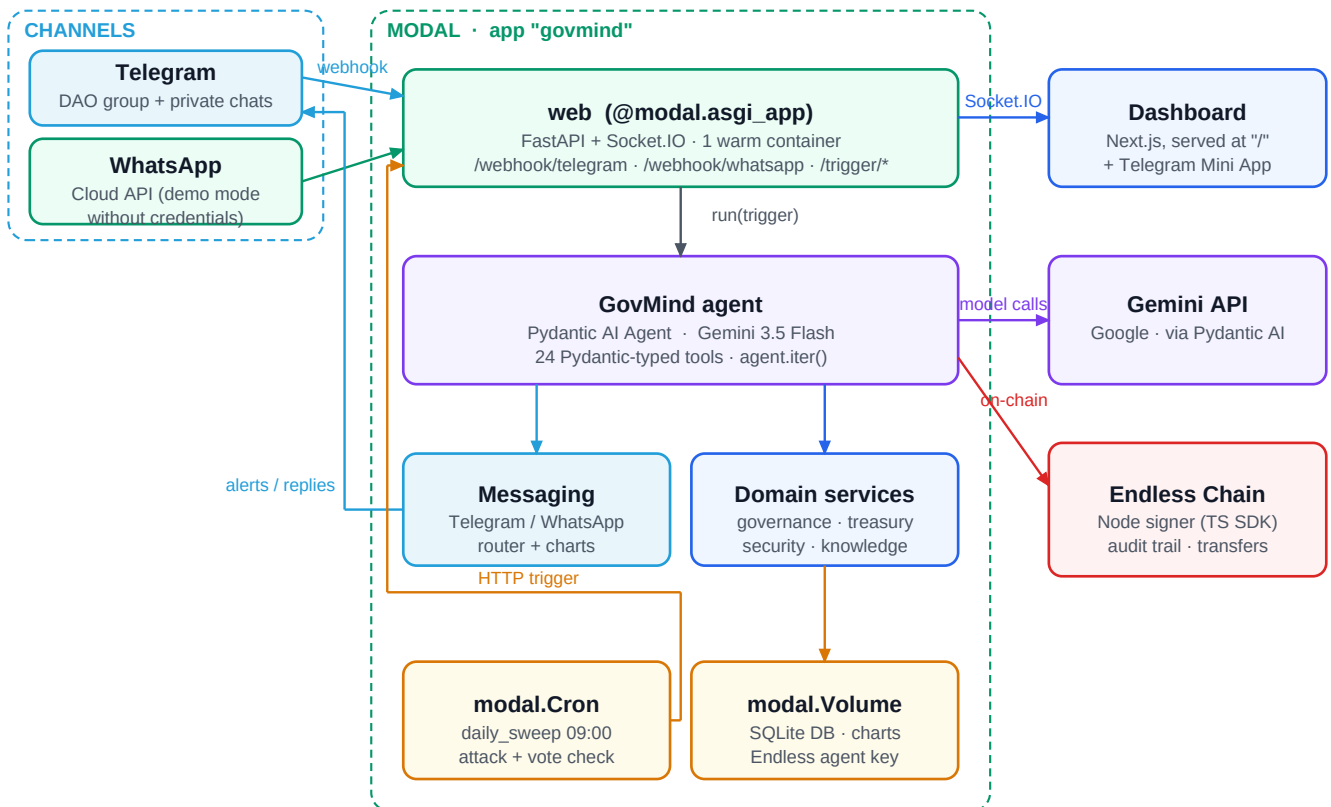


Figure 1. GovMind on Modal. Telegram and WhatsApp reach one FastAPI container; the agent calls Gemini and its tools, and streams every step to the dashboard.

Everything runs in a single Modal app. The **web** function serves the WhatsApp and Telegram webhooks, the REST and trigger API, the Socket.IO event stream and the static dashboard from one URL. The agent runs inside that same container, so its live events reach every connected dashboard and Mini App directly.

3. How the agent works

A run starts from a **trigger**: a Telegram or WhatsApp message, a dashboard button, a newly submitted proposal, or the daily cron sweep. Each trigger becomes a short natural-language task (for example, "A member just submitted Proposal #50, follow the PROPOSAL INTELLIGENCE process"), plus who sent it and the recent conversation.

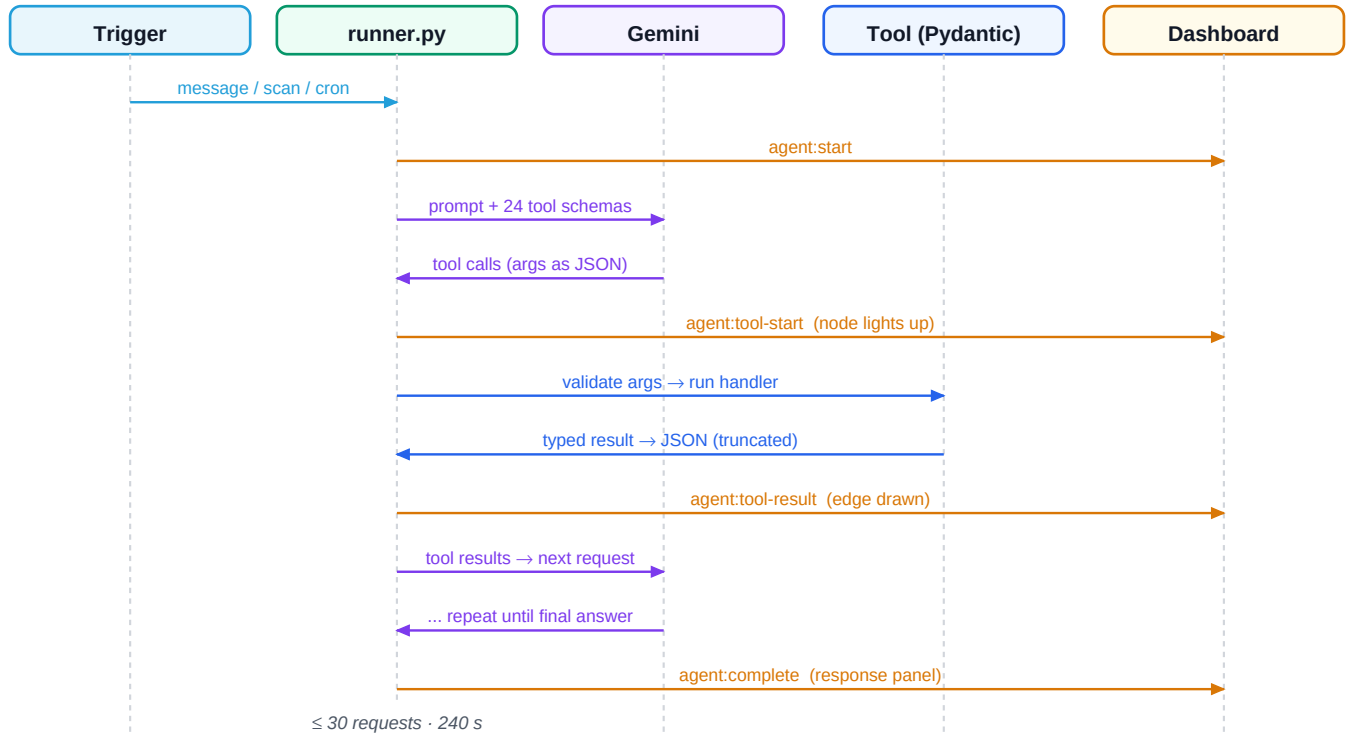


Figure 2. One agent run. The runner steps the Pydantic AI graph with `agent.iter()` and forwards each tool call and result to the dashboard.

3.1 The loop

- **Build.** `runner.build_agent()` creates a Pydantic AI Agent for this run: the GovMind system prompt, the Gemini model, and 24 tools. Tools are closures over a `RunContext` holding the sender and, for group messages, the group chat id.
- **Step.** `agent.iter()` walks the agent graph node by node. On each `CallToolsNode` the runner streams `FunctionToolCallEvent` / `FunctionToolResultEvent`, which become `agent:tool-start` and `agent:tool-result` `Socket.IO` events. That's what lights up nodes and draws edges in the graph.
- **Execute.** Every tool call goes through `tools.execute()`: validate arguments against the tool's Pydantic model, run the async handler, serialise the typed result to JSON and truncate it (at most 10 items, 300-character strings, 2,000 characters total) before it goes back to Gemini.
- **Finish.** When Gemini answers without tool calls, the answer becomes the run's response (`agent:complete`). Limits: 30 model requests, 240 seconds, and one run at a time (an `asyncio` lock) so the dashboard always shows a single coherent run.
- **Fail safe.** Any error (model, tool, timeout) ends the run with `agent:error` so the dashboard never hangs. Tool errors are returned to the model as data so it can recover or report them honestly.

3.2 The system prompt

The prompt (`agent/prompt.py`) defines GovMind's role and a fixed procedure for each capability, carried over from the original GovMind. For example, proposal intelligence is always: details → voting status → proposer wallet profile → proposer history (SQL) → treasury impact → post the briefing in a fixed template. Hard rules: never

fabricate data, never tell anyone how to vote, never transfer funds without an explicit request or a passed proposal, check team corrections first, and finish with log_action. Gemini runs with thinking set to "low", which keeps a full run within 13–37 seconds.

3.3 The 24 tools

Group	Tools	Backed by
Governance	get_active_proposals, get_proposal_detail, get_voting_status, get_member_vote_history, create_proposal, cast_vote, get_group_members, get_non_voters	SQLAlchemy async
Wallets & chain	link_wallet, get_wallet_for_user, get_onchain_balance, transfer_eds, record_action_onchain	Endless TS SDK via Node
Treasury	get_treasury_summary, get_treasury_transactions	SQLAlchemy async
Security	get_token_transfers, get_wallet_profile	SQLAlchemy async
Messaging	send_group_message (Telegram group + subscribers), send_direct_message	Telegram / WhatsApp
Utility	query_data (read-only SQL), generate_chart, get_knowledge, store_knowledge, log_action	SQL · matplotlib

4. How Pydantic is used

Pydantic is the contract layer everywhere data crosses a boundary: model ↔ tools, API ↔ dashboard, Telegram/WhatsApp ↔ backend, environment ↔ config. Invalid data is rejected at the edge with a clear error instead of failing deep inside the code.

4.1 Pydantic AI runs the agent

The agent is a `pydantic_ai.Agent` over a `GoogleModel` (Gemini) with `GoogleModelSettings` for the thinking level. Pydantic AI handles the provider protocol (function calling, Gemini thought signatures, retries), while GovMind steps the graph itself to stream events. The same code can switch to the Pydantic AI Gateway with one setting.

```
agent = Agent(
    GoogleModel("gemini-3.5-flash", provider=GoogleProvider(api_key=settings.gemini_api_key)),
    system_prompt=SYSTEM_PROMPT,
    tools=[Tool.from_schema(call, name=name, description=spec.description,
                             json_schema=spec.json_schema()) for name, spec in TOOLS.items()],
    model_settings=GoogleModelSettings(google_thinking_config={"thinking_level": "low"}),
)
async with agent.iter(prompt, usage_limits=UsageLimits(request_limit=30)) as run:
    async for node in run:
        if Agent.is_call_tools_node(node):
            async with node.stream(run.ctx) as events:
                await stream_to_dashboard(events)
```

4.2 Tool inputs: one model, two jobs

Each tool's arguments are a Pydantic model. Its JSON schema is what Gemini sees, and the same model validates what Gemini sends back before the handler runs. Field descriptions double as the documentation the model reads, `Literal` types become enums, `Field(gt=0)` becomes a constraint, and `extra="forbid"` rejects invented arguments.

```
class CastVoteArgs(ToolInput):
    # ToolInput: ConfigDict(extra="forbid")
    proposal_number: int = Field(description="The proposal number to vote on")
    voter_address: str = Field(description="Wallet address of the voter")
    vote: Literal["for", "against", "abstain"] = Field(description="The vote choice")
    voting_power: float | None = Field(None, description="Voting power (token balance). Default: 1")

@tool("cast_vote", "Casts a vote on an active proposal ...", lambda a: f'Casting vote "{a.vote}" ...')
async def _vote(args: CastVoteArgs, ctx: RunContext):
    result = await governance.cast_vote(**args.model_dump())
    return await _with_audit(result, "vote_cast", ...)
    # -> VoteCast | ToolError
    # adds the on-chain tx hash
```

If Gemini sends `vote="maybe"`, validation fails and the model gets back `"Invalid input for cast_vote: ..."`, so it corrects itself instead of corrupting data.

4.3 Typed results, webhooks and settings

- **Results** (schemas.py): every service returns a model such as `ProposalDetail`, `VotingStatus`, `WalletProfile`, `TreasurySummary` or `NonVoters`. Gemini and the REST API see the same shape. A shared validator normalises all datetimes to UTC.
- **Webhooks**: Telegram Update / Message / ChatMemberUpdated and WhatsApp WebhookPayload models parse incoming events, ignore unknown fields so platform changes don't break parsing, and expose helpers like `Chat.is_group`. Requests are authenticated first (Telegram secret-token header, WhatsApp X-Hub-Signature-256 HMAC).
- **API bodies**: every /trigger endpoint takes and returns a Pydantic model, which FastAPI validates and documents.
- **Settings**: `pydantic-settings` reads configuration from the environment or the Modal secret, and converts hosted Postgres URLs to the `asyncpg` driver automatically.

5. How Modal is used

Modal runs all of GovMind's compute from one Python file (backend/modal_app.py): no servers, Dockerfiles or tunnels. The public HTTPS URL it provides is used by Telegram's and Meta's webhooks, the dashboard, and the Telegram Mini App.

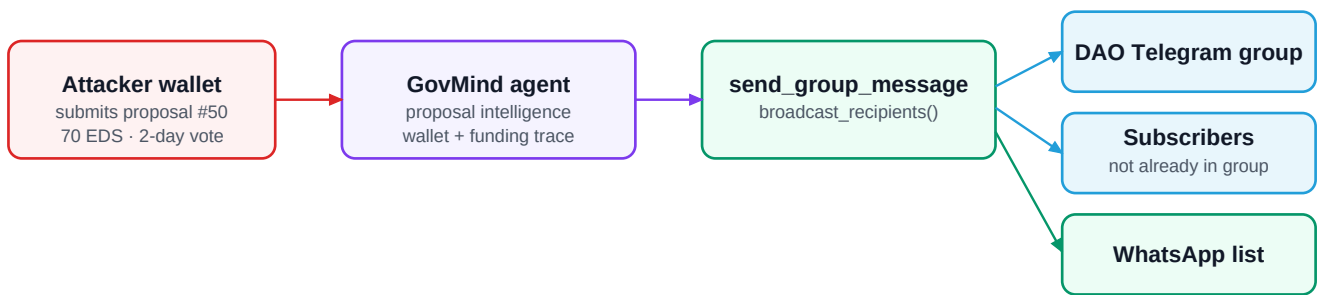
Modal feature	How GovMind uses it
modal.Image	Debian + Python 3.12 with uv-installed dependencies, Node 20 with the Endless TypeScript SDK (for signing), the govmind package, and the statically exported dashboard.
@modal.asgi_app web	FastAPI wrapped in a Socket.IO ASGI app: webhooks, /api, /trigger, /charts, and the dashboard at "/" plus the Mini App at "/miniapp/".
@modal.concurrent	Up to 100 concurrent requests on one container: webhooks, dashboard websockets and API calls while an agent run is in progress.
max_containers=1, min_containers=1	One always-warm container, so there are no cold starts during a live demo, and agent runs and websocket clients share in-process state.
modal.Volume govmind-data	Mounted at /data: the SQLite database (when no Postgres URL is set), generated chart PNGs, and the auto-generated Endless testnet key.
modal.Secret govmind-secrets	GEMINI_API_KEY, TELEGRAM_BOT_TOKEN, WhatsApp credentials, PUBLIC_BASE_URL; loaded into Pydantic settings at startup.
modal.Cron daily_sweep	Every day at 09:00 it calls the web app's attack-check and vote-check triggers (opt-in), so the runs appear live on the dashboard and alerts reach Telegram.
seed function	modal run modal_app.py::seed loads the MetaDAO demo data onto the volume.

On startup the web container creates tables, registers the Telegram webhook (setWebhook with a secret token) and sets the bot's menu button to open the Mini App. Deploys use deploy.sh: build the dashboard, stop the app, then modal deploy. The single container holds long-lived websockets, so stopping first swaps versions in about 30 seconds instead of waiting for sockets to time out.

6. Telegram: from Luffa to Telegram

The original GovMind lived in a Luffa group chat with a Luffa mini app. This version keeps the same model on Telegram:

Original (Luffa)	Now (Telegram)
Bot inside the DAO group chat; learns the group id	Add @GovMind_bot to a group; it learns the group from my_chat_member / messages and stores it
Tracks group members and @mentions	Tracks senders, new members and text mentions as tg:<id> members
Auto-links wallet addresses posted in chat	Same detection (hex or Base58 near "wallet")
Quick-chat buttons (link wallet)	Reply keyboard: Is proposal 49 safe? · Run attack scan · Treasury status · Who hasn't voted? · Link wallet · Check my EDS balance
Luffa mini app (DAO Health, Agent graph, floating chat)	Telegram Mini App at /miniapp/: DAO Health, live Agent graph, Feed and floating chat
Group messages → answer in the group	Same: questions in the group are answered in the group



Phones get a push notification: "GOVERNANCE ALERT" with evidence and a recommendation.

Figure 3. Alert fan-out. A group-triggered run answers only in that group; other alerts go to the DAO group, to private subscribers not already in it (checked with getChatMember, so no one gets it twice), and to WhatsApp.

7. Guardrails

- query_data runs agent-written SQL in a transaction that is always rolled back, and rejects write keywords and stacked statements.
- Webhooks are authenticated (Telegram secret token, WhatsApp HMAC signature); duplicate deliveries are ignored by message id.
- The prompt forbids telling members how to vote and forbids transfers without an explicit request or a passed proposal. transfer_eds requires a positive amount and a stated reason.
- On-chain calls check Endless RPC reachability first, and a circuit breaker skips the chain for 15 minutes after any timeout, so an outage never stalls a run. The agent reports the skipped step rather than inventing a transaction.
- Secrets live only in the Modal secret; HTTP client logging is silenced so bot tokens never appear in logs.

8. Demo flow (about 2 minutes)

Step	What happens	Where you see it
1	Alice asks "is proposal 49 safe?"; the agent investigates and replies	Graph animates; reply in feed
2	An attacker wallet submits proposal #50 (70 EDS); GovMind flags it	Alert on your phone
3	Attack scan traces the funding source and Sybil voting wallets	Second phone alert
4	Bob asks for a runway chart	Chart image in feed / chat

Source: github.com/Pranavreddy2007/tech-europe-hackathon